

06. 모의고사 — a!sk + 직무 기술면접 (설계 직무)

이 파일의 목적: "읽어서 아는 것"과 "시험장에서 말로 답하는 것"은 다르다. 이 문서는 실제 시험처럼 타이머를 켜고 푸는 모의고사다. 개념 정독은 `concepts/` 폴더, 매일 암기는 `daily_study/`, 키포인트 사전은 `03_interview_job_qbank.md`. 여기는 출제·연습·자가채점 전용.

지원 직무 = 설계(Design). JD 기준 디지털설계·RTL·SoC·검증·DFT·STA·CDC/RDC·SI/PI·고속 인터페이스가 핵심이다. 8대공정·소자물성·양산공정은 별도 직무군이므로 이 모의고사에서는 "기본 상식 라이트" 1문항으로만 다룬다(`concepts/11_공정_라이트.md`).

사용법(권장 루틴) 1. 각 SET를 실제 제한시간으로 푼다(아래 타이머 규칙). 답변은 반드시 소리내어 말하고 녹음한다. 머릿속으로만 하면 채점 불가. 2. 답변 후에야 그 문항의  채점 루브릭 (접힌 부분)을 펼쳐 자가채점한다. 먼저 보지 말 것. 3. 틀리거나 막힌 문항 → 우측 → 개념서 링크로 정독 후 재녹음. 4. SET 1회 = 1세션. 점수 기록표(맨 아래)에 회차별 점수를 적어 추세를 본다.

범례: ★기출 = 후기/공개자료에서 실제 출제가 확인된 유형(출처 `sources.md`, 본 문서 맨 아래 [출처]). ◆ 빈출예상 = JD 키워드·메모리 설계 필수 개념상 고확률.  = 관련 정독 문서.

채점 루브릭 공통 기준 (모든 문항 공통)

각 답변을 5점 만점으로 self-grade:

점수	기준
5	두괄식 결론 + 정확한 개념 + "왜"까지 + (직무/메모리)연결 또는 수치
4	결론·개념 정확하나 "왜" 또는 연결 1개 부족
3	핵심은 맞지만 두루뭉술 / 용어만 나열
2	방향만 맞고 내용 부정확
1	답 못 함 / 틀림

- 두괄식: 첫 문장에 답을 박는다. "결론부터 말씀드리면 ~입니다."
- 모르는 문항 대응 원칙: "모릅니다"로 끝내지 말 것. "로직 관점에서 이렇게 이해했고, 메모리에선 ~로 확장될 것 같습니다"로 추론을 보여라. (지원자 최강 무기 = 로직→메모리 브릿지)
- 목표: a!sk 문항 평균 4.0+, 기술면접 문항 평균 3.5+ (3회차 기준).

PART 1 — a!sk 모의고사 (AI 영상면접)

실전 포맷(후기 종합, 02_ask_prep.md · 00_overview.md 참조): 총 6~7문항. 문항당 준비 30~60초 → 답변 30초~최대 2~3분(유형별 상이). 면접관 없이 **자가 녹화 제출**. 평가는 사람이 **답변 내용**(논리·직무이해·문제해결) 중심.

확인된 구조 신호(공개 후기 기준): ①1분 자기소개 ②지원동기 ③일반 인성질문 ④직무 기초질문 ⑤상황질문(현업 상황+해결) ⑥상황질문(더 어려운 상황) — 3·4·5·6번은 **"사고를 묻는" 문항으로 준비시간이 길게(최대 3분) 주어진다**는 후기가 있다. 그래서 본 모의고사 a!sk SET에는 **상황질문(situational)**을 반드시 포함했다(기존 자료의 약점 보완).

타이머 규칙(연습용 고정): 자기소개/지원동기/인성 = 준비 30초·답변 60초. 직무기초 = 준비 30초·답변 60초. 상황질문 = 준비 60~90초·답변 90초. 실제 시험은 화면 안내를 따르되, 연습은 이 규칙으로 압축력을 기른다.

a!sk SET 1

Q1. (자기소개) 1분 자기소개를 해주세요. ◆ 준비 30s / 답변 60s

Q2. (지원동기) 여러 반도체 회사 중 왜 SK하이닉스 설계 직무에 지원했나요? ★ 준비 30s / 답변 60s

Q3. (인성) 팀으로 일하면서 갈등이 있었던 경험과, 그것을 어떻게 해결했는지 말해주세요. ★ 준비 30s / 답변 60s

Q4. (직무기초) DRAM은 왜 주기적으로 refresh를 해야 하나요? ★ 준비 30s / 답변 60s

Q5. (상황) 당신이 설계·검증한 디지털 IP가 시뮬레이션에서는 통과했는데, 실리콘(실제 칩)에서 간헐적으로 오동작합니다. 무엇부터 의심하고 어떻게 접근하겠습니까? ◆ 준비 90s / 답변 90s

Q6. (상황) 일정이 촉박한데, 검증 커버리지는 아직 목표에 못 미치고 매니저는 "이번 주에 sign-off 하자"고 합니다. 당신은 어떻게 판단·행동하겠습니까? ◆ 준비 60s / 답변 90s

Q7. (마무리) 마지막으로 하고 싶은 말을 해주세요. ◆ 준비 30s / 답변 45s

▼ ▣ SET 1 채점 루브릭 (답변·녹음 후 펼치기)

- **Q1 자기소개** 🔗 02_ask_prep § 2 , 04_resume_selfmastery
- 들어가야 할 것: ① 정체성 한 줄("설계 워크플로우의 false·낭비를 줄이는 엔지니어" 등) ② 대표 성과 1~2개 + 수치(RDC false -87.7% / EQ 1,760만 line / 사업부 첫 AI sign-off agent) ③ 설계 직무 연결한 문장.
- 감점: 경력 나열만 / 수치 없음 / 메모리 연결 없음 / 60초 초과.
- **Q2 지원동기** 🔗 05_positioning , concepts/10
- 들어가야 할 것: 끌림(메모리가 HBM·AI로 가장 빠르게 가치 성장) → 내 역량(디지털 설계·검증·sign-off) → "이 문제가 메모리 Peri·HBM Base Die 디지털 설계가 푸는 문제와 같다" 브릿지. **삼성 비방 금지**, 긍정 프레이밍.
- 감점: "큰 회사라서" / 추상적 / 직무(설계) 특정성 없음.
- **Q3 갈등** 🔗 02 § 2 , 04
- STAR: 상황→내 행동(가시화/RnR/소통창구)→결과. 예: 해외 연구소 kick-off 지연 → Jira로 task 가시화·PoC·RnR 명확화 → 일정 내 첫 양산 kick-off. **배운 점 한 줄**("가시화된 task + 명확한 책임자가 가장 빠른 협업 도구").
- 감점: 남 탓 / 갈등만 있고 해결·교훈 없음.
- **Q4 refresh** ★ 🔗 concepts/02_DRAM § refresh
- 핵심: 1T1C 셀의 **커패시터 전하가 누설로 소실** → 데이터 깨지기 전 **주기적 read+restore**. retention time(약 64ms/고온 32ms), tREFI(약 7.8μs), 미세화로 저장전하↓→retention 악화. (가능하면) refresh는 컨트롤러 FSM의 배경 housekeeping = 디지털 설계 영역.
- 감점: "전원 꺼지면 날아가서"(그건 휘발성 일반론, refresh 이유가 아님) / charge sharing과 혼동.
- **Q5 상황: 시물 통과·실리콘 오동작** ◆ 🔗 concepts/05_CDC_RDC , 06_STA
- 좋은 답의 뼈대: ① **간헐적**이라는 단서에 주목 → 타이밍/비동기 이슈 우선 의심: **CDC/RDC 메타스터빌리티**, hold violation, 클럭 skew/jitter, 비동기 reset. ② 환경 의존(전압·온도·코너) → STA corner/OCV 마진, IR drop. ③ 접근법: 재현조건 좁히기(전압/온도/주파수 sweep) → 정적분석(CDC/RDC/STA report) 재점검 → 해당 path의 constraint·동기화 구조 확인. ④ 브릿지: "제가 하던 CDC/RDC sign-off가 정확히 이 간헐 버그를 silicon 전에 잡는 일."
- 감점: "다시 시물 돌려본다"만 / 간헐성=타이밍/비동기 단서를 못 잡음.
- **Q6 상황: 커버리지 미달 sign-off 압박** ◆ 🔗 concepts/07_검증
- 좋은 답: 무작정 "안 됩니다"도, 무작정 "하겠습니다"도 아님. ① **리스크를 데이터로 가시화**(미달 커버리지가 어느 기능/시나리오인지, 그게 실패 시 영향), ② **우선순위화**(critical path·고위험 기능부터 커버리지 메우기), ③ 대안 제시(부분 sign-off + 잔여 리스크 문서화 + 후속 regression 계획), ④ 의사결정자에게 **판단 근거 제공**. "책임 회피"가 아니라 "근거 있는 합의". 자소서 톤(#프로야근러, 근거 있는 패기)과 정합.
- 감점: 무조건 거부 / 무조건 수용 / 리스크 정량화 없음.
- **Q7 마무리** 🔗 05
- 들어가야 할 것: 짧고 강한 한 방 — "로직과 메모리를 모두 본 시각으로, 설계 품질을 검증·자동화로 끌어올리겠다" + 입사 후 기여 의지. 45초 내.
- 감점: 같은 말 반복 / 늘어짐.

a!sk SET 2

Q1. (자기소개) 본인을 한 문장으로 정의하고, 그 근거가 되는 경험을 말해주세요. ◆ 준비 30s / 답변 60s

Q2. (지원동기) 입사 후 5년 안에 이루고 싶은 목표는 무엇인가요? ◆ 준비 30s / 답변 60s

Q3. (인성) 높은 목표를 세우고 도전했던 경험을 말해주세요. ★ 준비 30s / 답변 60s

Q4. (직무기초) CDC(Clock Domain Crossing)가 무엇이고 왜 위험한가요? ◆ 준비 30s / 답변 60s

Q5. (상황) 선배가 만든 기존 검증 환경이 비효율적이라고 느꼈습니다. 하지만 그 환경은 팀이 오래 써온 것입니다. 당신은 어떻게 하겠습니까? ◆ 준비 60s / 답변 90s

Q6. (상황) 처음 보는 IP의 버그를 맡았는데, 문서도 부족하고 설계한 사람은 퇴사했습니다. 어떻게 원인을 좁혀가겠습니까? ◆ 준비 90s / 답변 90s

Q7. (인성) 스트레스가 큰 상황을 어떻게 관리하나요? ★ 준비 30s / 답변 45s

▼ ■ SET 2 채점 루브릭

- **Q1** 🔗 04 : 한 문장 정의 + 그것을 증명하는 **단일 사례 + 수치**. "방법론만"이 아니라 "직접 설계도 했다" (SRAM 설계·DC/FC/PrimeTime) 근거 1개 곁들이면 겁 방어.
- **Q2** 🔗 01_team_fit , concepts/10 : 막연한 "전문가"가 아니라 **설계 직무 좌표**로. 예: "메모리 Peri/HBM Base Die 디지털 설계·검증의 sign-off를 책임지고, AI 자동화로 개발 TAT를 줄이는 사람". 회사 방향 (HBM·PIM)과 정합.
- **Q3 도전** ★ 🔗 02 § 2 : STAR + 수치. 예: 신입이 20여 명 얹힌 SFR flow를 IP-XACT로 전환 자청 → 1 IP 시연으로 합의 → TAT 12일→수초, error 47→3회. "근거 있는 패기".
- **Q4 CDC** ◆ 🔗 concepts/05_CDC_RDC
- 핵심: 서로 다른(비동기/위상차) 클럭 도메인 간 신호 전달 시 수신 FF의 **setup/hold 위반** → **메타스터빌리티(출력이 0/1 사이 진동)** → 임의값 **resolve** → 데이터 손상·reconvergence 불일치. 대응: 단일비트 2-FF 동기화기, 다중비트는 **gray code/async FIFO/handshake**. 신뢰성은 **MTBF**로 정량화.
- 감점: "클럭이 달라서 위험"만(왜=메타스터빌리티를 못 짚음).
- **Q5 상황: 비효율 기준환경 개선** ◆ 🔗 04 (자소서 도전과 정합)
- 좋은 답: ① **존중 + 데이터**("오래 썼다"를 인정하되 비효율을 정량 측정), ② 작은 PoC로 개선효과 시연 (전면 교체 X), ③ 팀 합의 후 점진 적용. 자소서3(신입이 표준 flow 자청 변경, 1 IP 시연으로 합의)과 같은 패턴.
- 감점: "그냥 바꾼다" / "참고 쓴다" 양 극단.
- **Q6 상황: 문서 없는 IP 디버그** ◆ 🔗 concepts/07_검증
- 좋은 답: ① 재현·격리(실패 케이스 최소화), ② 인터페이스/스펙부터 역추적(파형·로그·assertion), ③ 정적분석(Lint/CDC/STA)으로 구조적 의심지점 좁히기, ④ 이분탐색식으로 범위 축소. "사람이 없으면 신호와 데이터가 말하게 한다".
- **Q7 스트레스** ★ 🔗 02 § 2 : 짧고 건강하게(운동/취미) + 업무적 태도 1줄("문제를 task로 쪼개 통제 가능한 것부터"). 45초.

a!sk SET 3

Q1. (자기소개) 30초 안에 본인의 강점 2가지를 근거와 함께 말해주세요. ◆ 준비 30s / 답변 40s

Q2. (지원동기) 설계 직무에 필요한 역량은 무엇이라 생각하고, 본인은 그것을 어떻게 길렀나요? ★ 준비 30s / 답변 60s

Q3. (직무기초) 회로(또는 디지털 블록)를 설계할 때 고려해야 할 요소는 무엇인가요? ★ 준비 30s / 답변 60s

Q4. (직무기초) STA에서 setup violation과 hold violation의 차이는 무엇인가요? ◆ 준비 30s / 답변 60s

Q5. (상황) 당신이 제안한 설계 방식과 선배의 방식이 PPA(성능·전력·면적)에서 trade-off 관계입니다. 어떻게 결정을 이끌어내겠습니까? ◆ 준비 60s / 답변 90s

Q6. (상황) 양산 직전 칩에서 특정 조건(고온)에서만 fail이 발생합니다. 설계 검증 담당으로서 어떻게 원인을 분석하겠습니까? ◆ 준비 90s / 답변 90s

Q7. (인성) 본인의 약점은 무엇이고, 어떻게 보완하고 있나요? ★ 준비 30s / 답변 60s

▼ ▣ SET 3 채점 루브릭

- Q1 04 : 강점 2개 = (1) CDC/RDC·STA sign-off 방법론 (2) AI-EDA 자동화. 각각 수치 근거 1개. 40초 압축.
- Q2 설계 역량 ★ 01_team_fit : "정확한 개념 이해 + 검증/sign-off 사고 + 자동화" → 길러온 경험(직접 설계 + 방법론). "설계를 이해해야 좋은 검증/방법론이 나온다"는 역논리로 갭 방어.
- Q3 회로설계 고려요소 ★ concepts/01_소자 , 06_STA , 08_저전력
- 핵심: PPA(성능·전력·면적) trade-off를 축으로 + 타이밍 closure(setup/hold) + 신호/전력 무결성(SI/PI) + 신뢰성(aging/EM) + 테스트용이성(DFT) + 공정변이(PVT corner) + 제조가능성. 메모리는 특히 power-area-yield 민감. 학부 AES에서 면적 63%↓ = PPA 실전.
- 감점: "잘 동작하게"만 / PPA 언급 없음.
- Q4 setup vs hold ◆ concepts/06_STA
- 핵심: setup=데이터가 클럭 에지 전에 안정(느린 경로/max delay, 주파수 낮추면 해결, 다음 에지 기준). hold=에지 후에도 잠깐 유지(빠른 경로/min delay, 주파수 무관, 버퍼 삽입으로 수정, 같은 에지 기준). slack-skew로 마진. PrimeTime timing closure 경험 연결.
- 감점: 둘을 뒤집음 / "hold도 주파수로 고친다"(오답).
- Q5 상황: PPA trade-off 합의 ◆ concepts/06 , 08
- 좋은 답: ① 결정기준을 제품 요구사항으로 환원(이 제품에서 무엇이 1순위인가 — 속도? 전력? 면적/수율?), ② 두 안을 정량 비교 데이터로(타이밍 마진·전력·면적 수치), ③ 코너/리스크까지 보고 합의. "취향이 아니라 데이터·요구사항으로 결정".
- Q6 상황: 고온 fail 분석 ◆ concepts/08_저전력·신뢰성 , 02_DRAM
- 좋은 답: 고온 단서 → leakage↑/retention↓/타이밍 열화/IR drop 가설. ① 온도·전압 sweep으로 fail 경계 특성화, ② DRAM이면 retention/refresh 마진, 디지털이면 hot corner STA·hold·IR drop 점검, ③ 가설별로 분리 검증. "고온=leakage·retention·타이밍 마진" 연결고리를 보여라.
- Q7 약점 ★ 05 : 진술 + 보완 행동. 예: "메모리 디바이스 깊이가 로직 대비 얇다 → 그래서 2주간 DRAM/HBM/NAND를 1st-principles로 정리 중이고, 로직에서 쌓은 검증·sign-off가 그대로 메모리 Peri에 적용된다는 걸 확인했다." 약점→보완→오히려 강점 전환.

PART 2 — 직무 기술면접 모의고사 (대면 30분 시뮬레이션)

실전 포맷: 자소서·이력 기반 2:1 면접관, 전공·연구 꼬리질문, 약 30분. 핵심은 "정의"가 아니라 "왜"와 "꼬리질문 깊이까지 방어". 각 문항 뒤 L 꼬리 는 면접관이 실제로 파고드는 후속 질문이다 — 이것까지 답할 수 있어야 한다.

타이머 규칙(연습): 메인 질문 60~90초 답변 → 꼬리질문 각 30초 즉답. 한 SET(10문항)를 끝지 말고 연속으로(약 25~30분) 진행해 지구력을 기른다.

기술 SET A — 디지털 설계·검증·타이밍 (지원자 강점 영역, 깊게)

A1. ◆ CDC에서 단일 비트는 2-FF 동기화기로 충분한데, 다중 비트 신호는 왜 2-FF로 안 되나요? 어떻게 처리합니까?  꼬리: gray code를 쓰는 이유는? async FIFO는 포인터를 어떻게 동기화하나? handshake는 언제?  concepts/05_CDC_RDC

A2. ◆ RDC(Reset Domain Crossing)란 무엇이고 CDC와 무엇이 다른니까? 왜 tool에서 false violation이 폭증합니까?  꼬리: reset de-assertion이 왜 메타스터빌리티를 만드나? false violation을 어떻게 줄였나(본인 방법론)?  concepts/05_CDC_RDC , 04_resume_selfmastery

A3. ◆ hold violation은 왜 주파수를 낮춰도 안 고쳐지나요? 어떻게 수정합니까?  꼬리: clock skew가 setup과 hold에 주는 영향이 부호가 반대인 이유는? OCV/derating은 왜?  concepts/06_STA

A4. ◆ 합성 후 정합성 검증(Logic Equivalence Check)은 무엇이고, 시뮬레이션과 무엇이 다른니까?  꼬리: compare point가 뭔가? ECO/retiming 후 매칭이 왜 어려워지나? non-equivalence는 어떻게 디버그하나?  concepts/07_검증_DFT_MBIST

A5. ◆ UVM을 왜 쓰나요? constrained-random과 coverage-driven 검증을 설명해주세요.  꼬리: scoreboard 역할? functional vs code coverage 차이? coverage closure란? Custom UVM Agent를 spec에서 만든다는 게 무슨 의미?  concepts/07_검증_DFT_MBIST

A6. ◆ DFT의 scan과 ATPG를 설명해주세요. stuck-at과 transition fault의 차이는?  꼬리: scan shift/capture 동작? at-speed test는 왜 필요? scan shift 때 클럭이 CDC와 충돌하지 않나? scan compression?  concepts/07_검증_DFT_MBIST

A7. ◆ 메모리에서 MBIST와 redundancy/repair가 왜 중요한가요? (가장 메모리스러운 검증)  꼬리: March 알고리즘이 잡는 결함 유형? redundancy로 수율을 어떻게 끌어올리나? BIRA/BISR 흐름? on-die ECC는 왜 생겼나?  concepts/07_검증_DFT_MBIST , 08_저전력_신뢰성

A8. ◆ clock gating의 원리와 부작용은? STA·CDC 관점에서 무엇을 조심해야 합니까?  꼬리: ICG cell이 왜 glitch-free인가? enable 신호의 타이밍/CDC 위험? coarse vs fine-grain?  concepts/08_저전력_신뢰성

A9. ◆ 메타스터빌리티의 MTBF는 무엇에 의해 결정되며, 동기화기를 3단으로 늘리면 무엇이 좋아집니까?  꼬리: settling time과 주파수의 관계? 데이터율이 높아지면? 왜 무한히 단을 늘리지 않나?  concepts/05_CDC_RDC

A10. ◆ HBM Base Die / 메모리 Peri에는 여러 클럭·리셋 도메인이 존재합니다(코어/I/O/DLL·PLL/refresh). 디지털 설계·검증자로서 여기서 무엇을 가장 신경 쓰겠습니까?  꼬리: DLL과 PLL의 차이/용도? 데이터-스트로브(DQS) 타이밍이 왜 빠빠한가? 이 도메인 구조에서 CDC/RDC sign-off를 어떻게 설계하겠나?  concepts/05 , 06 , 09_고속인터페이스_PHY_SIP

▼ ■ 기술 SET A 핵심 채점 포인트 (요약 — 상세는 각 개념서)

- **A1 다중비트 CDC:** 비트별 도착 **skew** → 동기화 순간 비트마다 다른 값 샘플 → 잘못된 코드워드. 해법: 한 번에 1비트만 변하는 **gray code**(포인터), **async FIFO**(read/write 포인터 gray 동기화 + 메모리는 동기화 불필요), **handshake**(req/ack로 안정성 보장). 핵심: "데이터를 동기화하지 말고, **제어를 동기화**하라."
- **A2 RDC:** 비동기 reset **해제(de-assertion)** 시점이 수신 FF의 setup/hold를 위반 → CDC와 동일한 메타스터빌리티지만 트리거가 "클럭 에지"가 아니라 "reset 해제 순간". false 폭증 = tool이 모든 reset 시나리오를 보기 때문 → reset dependency 분석으로 실제 가능한 조합만 남김(본인 5단계 방법론: source 식별→의존성 시뮬→matrix→TCL constraint 자동생성, false -87.7%).
- **A3 hold:** hold는 **같은 에지**에서 데이터가 너무 빨리 바뀌어 생김(min delay) → 주기(T)와 무관 → 주파수를 낮춰도 그대로. 수정 = 빠른 경로에 **버퍼/delay 삽입**(단, setup을 깨지 않게 조심). skew 부호: 같은 skew가 setup엔 여유/hold엔 악화(또는 반대) — capture 클럭이 늦으면 setup 유리·hold 불리.
- **A4 LEC:** RTL↔netlist를 **formal하게 논리적 동일성 증명**(exhaustive, 벡터 불필요). compare point=두 설계의 대응 레지스터/출력. ECO·retiming은 레지스터 경계가 바뀌어 매칭점이 어긋남. 시뮬은 입력 벡터만큼만, LEC는 전 입력공간. (본인 Quick/Full EQ 이중화, 1,760만 line)
- **A5 UVM:** 재사용 가능한 계층형 testbench(agent=driver+monitor+sequencer) + CRV(제약 하 무작위로 코너 자동 탐색) + coverage(functional=의도한 시나리오 도달, code=코드 실행률). closure=목표 커버리지 충족. Custom UVM Agent=신규 프로토콜/IP의 driver·monitor·sequence를 스펙에서 직접 구현.
- **A6 DFT:** scan=FF를 shift register로 묶어 내부 상태를 직접 주입/관찰. shift(패턴 밀어넣기)→capture(1클럭 정상동작)→shift(결과 빼기). ATPG=fault를 깨우는 패턴 자동 생성. stuck-at(0/1 고정)·transition(at-speed, 속도 결함). scan shift 중 다중 클럭은 CDC와 충돌→clock control/제약 필요.
- **A7 MBIST/repair:** 내장 엔진이 March 패턴으로 어레이 self-test(인접셀 간섭·stuck·coupling 검출). 불량 셀을 **spare row/column**으로 대체(eFuse 기록) → 수율 구제. BISR=칩이 스스로 repair. on-die ECC=미세화로 늘어난 single-bit error를 칩 내부 정정(DDR5). "검증의 문제는 로직이나 메모리나 같다" 브릿지.
- **A8 clock gating:** 미사용 블록 클럭 차단으로 dynamic power↓. ICG cell=latch+AND로 enable을 클럭 low에 래치 → glitch-free. enable이 다른 도메인에서 오면 CDC 위험, enable 타이밍이 STA 대상.
- **A9 MTBF:** 메타스터빌리티가 시스템 고장으로 번지는 평균시간. 가용 **settling time**(클럭 주기 - 경로지연)에 지수적으로 의존, 주파수·데이터율↑이면 악화. 1단 추가=settling time 한 클럭 더 확보 →MTBF 지수적↑. 단을 늘리면 **latency 증가** trade-off라 보통 2~3단.
- **A10:** 도메인 경계의 CDC/RDC 무결성, DQS/스트로브 타이밍 마진(STA), reset 순서. DLL=지연 기반 클럭 정렬(주로 메모리 I/O 위상 맞춤), PLL=주파수 합성/체배. 브릿지: "내가 로직에서 한 다중도메인 sign-off가 그대로 핵심."

기술 SET B — 메모리 디바이스·HBM·소자·산업 (갭 보강 영역)

B1. ★ DRAM과 NAND Flash의 근본적 차이를 아는 만큼 설명해주세요. ㄱ, 꼬리: 왜 DRAM은 커패시터를 쓰냐? NAND가 3D로 간 이유는? DRAM도 3D로 갈 수 있나? [🔗 concepts/02_DRAM](#) , [03_NAND](#)

B2. ★ DRAM을 설명해주세요 — 셀 구조부터 읽기 동작까지. ↳,꼬리: 왜 read가 파괴적인가? restore는 왜 필요? sense amp는 무엇을 하나? 왜 BL을 VDD/2로 precharge하나? 🔗 [concepts/02_DRAM](#)

B3. ★ NAND를 설명해주세요 — 어떻게 비휘발성으로 데이터를 저장합니까? ↳,꼬리: floating gate vs charge trap? 왜 read/program은 page, erase는 block 단위? P/E cycle 수명은 왜 생기나? 🔗 [concepts/03_NAND](#)

B4. ★ HBM은 왜 어렵습니까? (공정·패키징·검증 병목) ↳,꼬리: TSV가 와이어본딩 대비 장점? MR-MUF의 차별점? 스택 수율이 왜 곱셈인가? base die를 로직 파운드리로 가져가는 의미? 🔗 [concepts/04_HBM_패키징](#)

B5. ★ 신호무결성(SI)과 전력무결성(PI), 어떻게 대응합니까? HBM 2048-bit I/O에서 왜 더 어렵나요? ↳,꼬리: crosstalk를 줄이는 레이아웃 기법? termination 종류? decap의 역할? eye diagram이란? 🔗 [concepts/09_고속인터페이스_PHY_SIPI](#)

B6. ★ MOSFET의 동작 원리를 설명해주세요. ↳,꼬리: triode와 saturation 경계 조건? Vth를 결정하는 요소? body effect? 🔗 [concepts/01_소자_MOSFET](#)

B7. ★ Short Channel Effect(SCE)를 설명해주세요. (확인된 단골 기출) ↳,꼬리: DIBL이 정확히 무엇? FinFET/GAA가 SCE를 어떻게 완화? SCE가 DRAM retention과 어떻게 연결되나? 🔗 [concepts/01_소자_MOSFET](#)

B8. ◆ DRAM의 RowHammer란 무엇이고 어떻게 대응합니까? NAND의 read disturb와 어떤 공통점이 있나요? ↳,꼬리: 왜 인접 row가 영향받나? TRR 동작? DDR5에서 강화된 점? 보안 관점? 🔗 [concepts/08_저전력_신뢰성](#)

B9. ★ HBM 시장에서 SK하이닉스의 위상은 어떻습니까? HBM4는 무엇이 다른가요? ↳,꼬리: SK하이닉스가 선두가 된 이유? 삼성·마이크론과의 차이? 2048-bit·base die 변화의 의미? 커스텀 HBM이란? 🔗 [concepts/10_산업_AI메모리_PIM](#)

B10. ◆ PIM(Processing-in-Memory)이란 무엇이고, 왜 주목받나요? SK하이닉스 제품은? ↳,꼬리: 메모리 월을 어떻게 푸나? 유리한 워크로드는? 상용화 과제는? (이게 설계 직무와 어떻게 연결되나?) 🔗 [concepts/10_산업_AI메모리_PIM](#)

▼ ■ 기술 SET B 핵심 채점 포인트 (요약 — 상세는 각 개념서)

- **B1:** DRAM=휘발성·1T1C·랜덤액세스·고속·refresh 필요·메인메모리 / NAND=비휘발성·전하저장 (FG/CTF)·string 직렬·page read·block erase·P/E 마모·스토리지. 4축(휘발성/셀구조/접근단위/속도·내구·용도). 둘 다 **셀 바깥 Peri/컨트롤러 디지털 로직**이 존재 = 내 영역.
- **B2:** WL=access TR 게이트, BL=전하 통로. 읽기: BL을 **VDD/2 precharge**(0/1 대칭 마진+저전력) → WL on → 셀캡과 BL 기생캡 **charge sharing**(±수십mV) → sense amp(cross-coupled latch) 차동 증폭 → 읽으면 셀 전하 깨짐(**destructive**) → **restore** 필수.
- **B3:** 절연막(floating gate/charge trap)에 갇힌 전자가 TR의 Vth를 바꿔 1/0. program=FNE터널링 전자 주입(ISPP+verify), erase=block 단위 전자 방출. string 직렬 구조 때문에 read/program은 page, erase는 block. P/E 반복→tunnel oxide 열화→수명 한계.
- **B4 HBM:** DRAM die 8~16단 적층+TSV+base die, 2.5D 인터포저로 GPU 연결. 난점: TSV 정렬·신뢰성, 적층 본딩(MR-MUF로 warpage/열 제어), **스택 수율=각 die 수율의 곱**(KGD 필수), 가운데 die 열, 적층 후 내부 접근 곤란+초광폭 I/O SI. base die를 로직 공정으로=연산·커스텀·전력.
- **B5 SI/PI:** SI=신호 왜곡(reflection/crosstalk/ISI/jitter) → 임피던스 매칭/termination·차동·간격·차폐·등화·eye 관리. PI=전원 안정(IR drop/SSN/ground bounce) → PDN·decap·power grid. 2048-bit·고속에서 동시 스위칭·결합 폭증.
- **B6 MOSFET:** 게이트 전압이 gate-oxide 커패시터로 채널에 전하 유도, **Vgs>Vth에서 반전층 형성** → Vds로 source-drain 전류. 영역: cut-off/triode/saturation. 경계 Vds vs Vgs-Vth. (소자 이해 = STA cell delay·전력 모델의 토대)
- **B7 SCE:** 채널이 짧아져 **drain 전계가 채널을 직접 제어→게이트 제어력 약화** → Vth roll-off, DIBL, punch-through, velocity saturation, leakage↑. 대응 FinFET/GAA(게이트가 채널을 더 감쌈). DRAM access TR의 leakage↑→retention 약화→refresh 부담.
- **B8 RowHammer:** 특정 row 반복 access→인접 row 전하 누설 가속→bit flip(신뢰성+보안). 대응 TRR/추가 refresh, DDR5 강화. NAND read disturb와 "반복 access→인접 셀 교란→모니터·완화"가 대칭. 둘 다 디지털 제어 로직+검증.
- **B9:** "SK하이닉스가 HBM 선두"(점유율 구체수치는 변동/추정 — 단정 X). 이유=first-mover·MR-MUF·엔비디아 협업. HBM4=2048-bit, per-pin 속도↑(변동), 16Hi, base die 로직 파운드리화·커스텀 HBM. 톤=균형(삼성 비방 금지).
- **B10 PIM:** 메모리 내부에 연산 내장→CPU/GPU로의 데이터 이동(메모리 월)↓→대역폭·전력 효율↑. memory-bound·LLM decode에 유리. SK하이닉스 GDDR6-AiM/AiMX. 과제=SW 스택·표준. 설계 연결 = "메모리 안의 디지털 연산 로직"=설계·검증 수요(내 디지털+검증+AI 교집합).

기술 SET C — RTL·SoC·버스·컨트롤러 (설계 직무 정조준, JD 키워드)

JD에 반복 등장하나 기존 자료에 약했던 영역을 정조준한 세트. 지원자의 "방법론·검증만이 아니라 직접 설계도 한다"를 증명하는 구간. [concepts/12](#) · 13 · 14 가 정독 짝.

C1. ◆ 동기 설계가 비동기 설계보다 주류인 이유는? 단점은 무엇으로 메웁니까? [_, 꼬리: 비동기의 장점(저전력·평균성능)? 동기가 STA로 검증 쉬운 이유? 글로벌 클럭의 부담은? [concepts/12_RTL_디지털설계_기초](#)]

C2. ◆ **Moore와 Mealy FSM의 차이**는? 어떤 상황에 무엇을 쓰나요? L,꼬리: 출력 glitch가 문제될 때? one-hot encoding의 장단점? 안전한 FSM(미정의 상태 복구)? [concepts/12_RTL_디지털설계_기초](#)

C3. ◆ Verilog에서 **blocking(=)**과 **non-blocking(<=)**의 차이와, 잘못 쓰면 생기는 문제는? L,꼬리: 순차로직에 왜 non-blocking? simulation-synthesis mismatch 사례? 의도치 않은 latch 추론은 왜? [concepts/12_RTL_디지털설계_기초](#)

C4. ◆ **파이프라이닝**은 무엇을 개선하고 무엇을 못 합니까? hazard에는 어떤 종류가 있나요? L,꼬리: throughput vs latency? RAW/WAR/WAW 중 진짜 위험한 것? forwarding/stall? 너무 깊게 쪼개면? [concepts/12_RTL_디지털설계_기초](#)

C5. ◆ 버스의 기본인 **valid/ready 핸드셰이크**와 **backpressure**를 설명하세요. L,꼬리: ready가 valid에 의존하면 안 되는 이유(조합 루프/deadlock)? skid buffer는 왜? [concepts/13_온칩버스_인터커넥트](#)

C6. ◆ **AXI가 AHB보다 좋은 점**은? (5채널·outstanding·burst·out-of-order) L,꼬리: 채널을 왜 분리하나? outstanding이 성능에 주는 이점? ID 기반 out-of-order? APB는 언제? [concepts/13_온칩버스_인터커넥트](#)

C7. ◆ 인터커넥트에서 **arbitration(중재)**이 왜 필요하고, round-robin이 fixed-priority보다 나은 점은? L,꼬리: starvation이란? credit-based flow control? deadlock을 어떻게 막나? [concepts/13_온칩버스_인터커넥트](#)

C8. ◆ **메모리 컨트롤러**가 하는 일을 설명하세요. (명령 변환·타이밍 집행·스케줄링) L,꼬리: open-page vs closed-page 정책? FR-FCFS? tRCD/tRP 같은 타이밍을 누가 지키나? bank 병렬성을 어떻게 활용? [concepts/14_메모리컨트롤러_RAS](#) , [concepts/02_DRAM](#)

C9. ◆ **RAS**란 무엇이고, ECC 계층(on-die / link / system)은 어떻게 다른가요? L,꼬리: scrubbing(patrol/demand)이란? CE vs UE? 컨트롤러 레벨 RowHammer 완화? 왜 데이터센터에서 필수? [concepts/14_메모리컨트롤러_RAS](#) , [concepts/08_저전력_신뢰성](#)

C10. ◆ **async bridge**(버스가 클럭 도메인을 건널 때)는 무엇이고 왜 까다롭나요? L,꼬리: 이게 CDC와 같은 문제인 이유? async FIFO로 어떻게 푸나? reset 도메인은(RDC)? [concepts/13_온칩버스_인터커넥트](#) , [concepts/05_CDC_RDC](#)

▼ ■ 기술 SET C 핵심 채점 포인트

- **C1:** 동기=공통 클럭 → STA로 정적 타이밍 검증 가능·설계/검증 단순(주류). 비동기=글로벌 클럭 없어 평균 성능·저전력 유리하나 hazard·검증 난해. 단점(클럭 분배·전력)은 CTS·clock gating으로 완화.
- **C2:** Moore=출력이 현재 state만(registered, glitch 없음, 1-cycle latency). Mealy=출력이 state+입력 (빠른 반응, state 적음, 입력 glitch 전파). 출력 안정성 필요→Moore, 빠른 반응/면적→Mealy. one-hot=빠르고 디코딩 단순, FF 더 씬.
- **C3:** blocking(=)=순차적 즉시 대입(조합로직 always_comb), non-blocking(<=)=동시 갱신(순차로직 always_ff). 순차에 blocking 쓰면 race-shift 오류, 조합에 불완전 sensitivity/latch 추론→sim-synth mismatch.
- **C4:** 파이프라인=조합경로를 레지스터로 분할→throughput(주파수)↑, latency는 유지/증가(개선 못 함). hazard: 구조/데이터(RAW read-after-write)/제어. forwarding·stall·bubble. 너무 깊으면 레지스터 오버헤드·latency·분기 페널티↑.
- **C5:** valid(송신 준비)·ready(수신 준비) 둘 다 1인 사이클에만 전송. ready를 내려 backpressure. ready가 valid의 조합함수면 루프/타이밍 문제→skid buffer로 끊음.
- **C6:** AXI=AR/R/AW/W/B 5채널 독립(읽기/쓰기 동시)·outstanding(여러 미완료 거래로 지연 은닉)·burst·ID로 out-of-order 완료. AHB=단일 파이프라인. APB=저속 주변장치 단순 제어.
- **C7:** 여러 마스터가 한 자원 경쟁→중재 필요. fixed priority=낮은 우선순위 starvation 위험. round-robin=공정. credit-based=수신 여유만큼만 발행해 overflow/deadlock 방지.
- **C8:** 호스트 요청을 JEDEC 명령(ACT/RD/WR/PRE/REF)으로 변환+주소 디코딩, tRCD/tRP/tRAS 등 타이밍을 위반 없이 발행, 스케줄러가 bank/bank-group 병렬성으로 명령 인터리빙. open-page=지역성 베티, closed-page=충돌 줄임. FR-FCFS=준비된(row hit) 요청 우선.
- **C9:** RAS=Reliability/Availability/Serviceability. on-die ECC=셀 단위(미세화 보정), link/HBM ECC=전송 오류, system ECC=rank 단위 전체. scrubbing=잠자는 비트오류를 주기적/접근시 정정. CE=정정됨, UE=정정불가. 컨트롤러가 TRR/RFM으로 RowHammer 완화.
- **C10:** 버스가 서로 다른 클럭 도메인을 건너면 메타스터빌리티 위험=CDC 문제. async FIFO(gray 포인터 동기화)로 데이터, handshake로 제어. reset 도메인 다르면 RDC도.

PART 3 — 빠른 구술 드릴 (랜덤 30초 답변, 워밍업/마무리용)

타이머 30초. 정의→핵심→(가능하면 메모리 연결) 한 호흡. 매일 5개씩 무작위로.

1. precharge를 VDD/2로 하는 이유? ◆
2. tRCD·tRP·tRAS·tRC의 관계? ◆
3. self-refresh와 auto-refresh의 차이? ◆
4. bank와 bank group이 필요한 이유? ◆
5. CMOS가 정적 전류가 거의 0인 이유? ★
6. leakage의 종류(subthreshold/gate/junction/GIDL)? ◆
7. DLL과 PLL의 차이? ◆
8. on-die ECC가 필요해진 이유? ◆

9. DFE/CTLE는 무엇을 보상하나? ◆(JD 키워드)
10. ODT/termination이 필요한 이유? ◆(JD 키워드)
11. scan compression의 목적? ◆
12. formal verification이 잘 맞는 문제 유형? ◆
13. PVT corner를 보는 이유? ◆
14. NAND에서 SLC/MLC/TLC/QLC의 trade-off? ★
15. 3D V-NAND로 간 이유? ★
16. 8대공정 순서를 한 줄로(교양 수준)? ★ → [concepts/11_공정_라이트](#)
17. EM(electromigration)이 power/clock net에서 특히 중요한 이유? ◆
18. UPF로 무엇을 기술하나? ◆
19. AMBA/AXI 같은 on-chip 버스를 쓰는 이유? ◆(JD 키워드)
20. CDR(Clock Data Recovery)이 무엇을 하나? ◆(JD 키워드)
21. NAND read/program/erase 레이턴시 순서와 이유? ◆
22. throughput과 latency의 차이(메모리 관점)? ◆
23. 노이즈 마진(NMH/NML)이란? CMOS가 큰 이유? ★
24. open-page vs closed-page 정책? ◆(JD 키워드)
25. reset synchronizer가 필요한 이유? ◆
26. FR-FCFS 스케줄링이란? ◆

답이 막히는 번호 = 그날의 정독 우선순위. 모든 답은 [concepts/](#) · [daily_study/](#) · 03 에 있음.

PART 4 — RTL 설계/코딩 문제 (설계 직무 화이트보드 대비)

설계 직무 면접은 "설명"뿐 아니라 "**이걸 Verilog로/상태도로 그려봐**" 같은 즉석 설계를 시키기도 한다(특히 RTL/SoC/Logic 롤). 화이트보드/구술로 접근법 → 블록도/상태도 → 핵심 코드 골격 → 코너 케이스를 말하는 연습. 완성 코드보다 **사고 과정과 함정 인지**가 평가된다. 정독 짝: [concepts/12 · 13 · 05](#).

연습법: 문제당 5분. 종이에 (1) 입출력 정의 (2) 블록도/상태도 (3) 핵심 always 블록 (4) 리셋·코너. 그 뒤 ■ 펼쳐 자가점검.

- P1.** 2-FF synchronizer를 Verilog로 작성하고, 왜 2단인지·언제 3단으로 늘리는지 설명하라. ◆ **P2.** 비동기 assert·동기 deassert "reset synchronizer"를 설계하라. 왜 deassert만 동기화하나? ◆ **P3.** mod-N 카운터와 ÷3 클럭 분주기(50% duty)를 설계하라. (홀수 분주의 함정) ◆ **P4.** 입력 비트스트림에서 "1011"을 검출(overlap 허용) 하는 FSM을 상태도+코드로. Moore/Mealy 중 선택·이유. ◆ **P5.** gray code 카운터를 설계하라. 왜 CDC 포인터에 gray를 쓰나? ◆ **P6.** async FIFO의 구조(read/write 포인터 gray 동기화)와 깊이 계산 개념을 설명하라. ◆ **P7.** round-robin arbiter(N요청)를 설계하라. fixed-priority 대비 차이는? ◆ **P8.** valid/ready 스킷 버퍼(skid buffer) 로 backpressure를 끄는 구조를 설계하라. ◆ **P9.** edge detector(rising) 및 느린→빠른/빠른→느린 펄스 동기화기를 설계하라. ◆ **P10.** glitch-free clock gating(ICG) 동작과, 왜 raw AND 게이팅이 위험한지 설명하라. ◆

▼ ▣ PART 4 핵심 채점 포인트 (코드 골격·함정)

- **P1:** $q1 \leq d$; $q2 \leq q1$; (non-blocking). 첫 FF가 메타 상태를 settle할 1클럭을 벌어줌. 데이터율↑/MTBF 부족 시 3단. 단일 비트에만, 다중비트 금지.
- **P2:** `rst_sync` 를 두 FF로 — reset assert는 비동기로 즉시(전원/이상 시 즉각 정지), deassert는 클럭에 동기화(메타·recovery/removal 회피). 두 FF에 async reset 입력, 입력 D는 1로 묶음.
- **P3:** 짝수 분주=카운터 MSB/토글. **홀수(÷3) 50% duty**=상승엣지 카운터와 하강엣지 카운터를 OR(또는 1.5클럭 위상). 함정: 단순 카운터는 듀티 비대칭.
- **P4:** 상태 `S0..S`(검출). overlap이면 부분일치로 되돌아감(예 "1011" 검출 후 마지막 "1"을 다음 후보로). Moore=검출 출력이 한 박자 늦지만 glitch 없음. 코드: `always_ff` 상태천이 + 출력 디코드.
- **P5:** 인접 코드가 **1비트만** 변함 → CDC 동기화 시 한 번에 한 비트만 흔들려 안전. async FIFO read/write 포인터에 사용.
- **P6:** write/read 포인터를 각각 gray로 변환→상대 도메인 2-FF 동기화→full/empty 비교. 깊이=버스트·생산/소비율·동기화 지연을 건디게(=유실 없는 최소 깊이). 메모리 자체는 동기화 불필요(주소만).
- **P7:** 마지막 grant 다음부터 순환 우선순위(mask+priority encoder 또는 rotating pointer). fixed-priority의 starvation 제거.
- **P8:** 두 단 레지스터로 valid/data를 잡아 ready 하강에도 1개 더 받아냄 → 조합 ready 루프 차단·full throughput. (간단형: skid register + 상태)
- **P9:** $\text{edge detect} = d \ \& \ \sim d_delayed$ (1클럭 지연 후 비교). 느린→빠른=2-FF 동기화 후 edge. 빠른→느린=토글+동기화+edge(펄스 폭이 수신 클럭보다 짧으면 유실되니 toggle 방식).
- **P10:** ICG=enable을 클럭 **low 구간**에 latch한 뒤 AND → enable이 클럭 high 중 흔들려도 출력 glitch 없음. `raw_clk & en` 은 en 천이가 클럭 high에 겹치면 글리치 클럭 발생→치명적.

⚠ 이 문제들은 "정답 코드 암기"가 목적이 아니다. **말로 설계 과정을 풀어내는 연습**이 핵심 — `concepts/12 · 13` 을 먼저 정독한 뒤 도전하라.

점수 기록표 (회차별 추세 관리)

날짜	SET	평균점수(/5)	4점 미만 문항#	재학습 완료
	a!sk SET1			
	a!sk SET2			
	a!sk SET3			
	기술 SET A			
	기술 SET B			
	기술 SET C			
	PART4 설계문제			

목표: 3회차에 a!sk 평균 4.0+, 기술 3.5+. 같은 문항을 2주 간격 2회 이상 풀어 "외운 답"이 아니라 "이해한 답"인지 확인.

[출처] — 실제 기출/후기 확인 (★ 표시 근거)

a!sk 구조 및 직무 기출(SCE·포토·식각·DRAM·NAND·8대공정·MOSFET·회로설계 고려요소·신호무결성·HBM 시장)은 아래 공개 후기/가이드에서 교차 확인. 상세 링크는 [sources.md](#) 참조.

- 원스펙: SK하이닉스 면접 기출 2024~2025 / 25하 직무면접 기출·전략
- 링커리어: 2026·2025 면접 합격후기(질문리스트), DRAM 회로설계 직무 후기
- 잡플래닛: a!sk 면접 질문·후기 / 2026 실제 질문 모음
- ecareer: a!sk 전형 분석 / 면접 합격전략(기출 총정리)
- welldone-interview: SK하이닉스 면접 질문 50선
- SK Careers Journal: 8대공정·DRAM 설계 직무 / 반도체 8대공정 요약

⚠ 후기 기반 정보는 시준·직무·면접관별 편차가 크다. **확정 기준은 본인 지원 공고와 실제 응시 화면 안내.** ★는 "이런 유형이 실제로 나왔다"는 신호이지, 토씨 그대로의 출제 보장이 아니다.